# Few-View Computed Tomography — Requirements and Restrictions

Computational, software and reproducibility conditions for the supplied CT benchmark

## 1. Scope

This document records the conditions and restrictions that matter when reproducing the CT benchmark. It is intended to prevent a technically successful Python run from being mistaken for a faithful reproduction when important problem or computational settings have been changed.

## 2. Core benchmark definition

| Parameter | Required benchmark value |
|---|---|
| Image dimensions | 1024 x 1024 |
| Number of unknowns | 1,048,576 |
| Number of views | 64 |
| Measurements | 65,536 |
| Measurement-to-unknown ratio | 0.0625 |
| CT geometry | Parallel-beam |
| Operator representation | Matrix-free |
| Reference image | Shepp-Logan-type synthetic phantom |
| Noise level | 0.002 |
| Regularisation values | 1e-6, 1e-5, 1e-4, 1e-3, 1e-2 |
| Solvers | CG, PCG, LSQR, LSMR |
| Tolerance | 1e-6 |
| Maximum iterations | 120 |
| Repeats | 2 |

These values come directly from the supplied CT implementation and the dissertation description. They define the computational experiment. Changing them is legitimate for a new experiment, but the resulting data should not be labelled as the original benchmark results.

## 3. Matrix-free restriction

The CT experiment must be understood as a matrix-free calculation. The supplied implementation uses scipy.sparse.linalg.LinearOperator to represent the CT forward and adjoint operations. The code rotates the image to generate projection data and performs the corresponding backprojection for the adjoint operation. It does not build and store an explicit CT system matrix.

This is not merely an implementation preference. The matrix-free formulation is part of the point of the experiment: the benchmark examines iterative methods on a large-scale CT inverse problem without materialising the full system matrix.

## 4. Memory requirements

The calculation is large enough that memory availability must be considered before submission. A direct-solver implementation is not appropriate for this CT problem simply because a large-memory node is available: the problem has 1,048,576 unknowns and the CT operator is intentionally handled matrix-free.

For the direct-solver cases in the wider dissertation experiments, allocations below 128 GB were found to fail. This should be treated as a practical warning about direct methods in this project, not as a claim that 128 GB is a universal minimum for every implementation. For the CT benchmark, memory requirements depend on the matrix-free implementation, Python/SciPy versions and execution environment.

Before running the benchmark on a University cluster, the researcher should check the University's current memory-allocation policy and the memory limits of the selected queue or partition. A successful small test is not evidence that the full 1024 by 1024 experiment will fit in the same allocation.

## 5. CPU requirements

CPU allocation is not fixed by the Python file. The appropriate number of CPUs depends on the machine, queue policy and environment in which the researcher runs the benchmark. The current University documentation should therefore be checked before requesting resources.

The CPU count should be recorded alongside the results because it affects wall-clock timing and makes later comparisons more meaningful. Two runs with identical mathematical parameters but different hardware or CPU allocation can produce different runtimes.

## 6. Slurm requirements

The large-scale benchmark should be treated as a batch calculation on a Slurm-managed system when local resources are insufficient. The user must learn the current CSF login procedure from University documentation. Account names, partitions, module names and environment commands are infrastructure details and are intentionally not fixed in this repository document.

The dissertation-style allocation used a 24-hour wall-time request. A corresponding template is shown in the run-instructions document. The exact memory and CPU request should be checked against the current University's allocation rules. In particular, CPU allocation must not be copied blindly from an old job because cluster policies and available resources can change.

## 7. Software requirements

The supplied programs use Python and the following scientific Python components: NumPy, SciPy, pandas and Matplotlib. CT.py also uses SciPy's ndimage and sparse LinearOperator/iterative-solver functionality. The plotting script uses pandas, Matplotlib and pathlib.

| Component | Role in the benchmark |
|---|---|
| Python 3 | Runs both programs. |
| NumPy | Numerical arrays, random noise and norms. |
| SciPy | CT image rotation, LinearOperator and iterative solvers. |
| pandas | CSV creation, aggregation and plotting-data preparation. |
| Matplotlib | Creation of PNG figures. |

The precise Python and package versions used for a future run should be recorded. Different versions can change numerical behaviour or performance even when the source code is unchanged.

## 8. Randomness and repeatability

The benchmark uses explicit random seeds. The main seed is 20260904, and the preconditioner uses a separate fixed seed of 12345. The noise is generated from a normal distribution and scaled to the requested noise level relative to the clean projection norm.

Keeping the supplied seeds is important if the intention is to reproduce the same generated numerical inputs. Changing the seed creates a different noisy data set and can therefore change solver residuals, reconstruction errors and runtimes.

## 9. Solver restrictions

The four solver labels have specific meanings in the supplied implementation. CG is applied to the regularised normal operator. PCG applies CG with the supplied diagonally estimated preconditioner. LSQR and LSMR solve the augmented least-squares formulation.

The benchmark should not be modified by silently replacing one method with a different implementation while retaining the same solver label. If a different library implementation or preconditioner is investigated, that should be stated explicitly.

## 10. Regularisation sweep

The supplied sweep contains exactly five values:

| Run order | lambda |
|---|---|
| 1 | 1e-6 |
| 2 | 1e-5 |
| 3 | 1e-4 |
| 4 | 1e-3 |
| 5 | 1e-2 |

The sweep is important because the dissertation discusses how runtime and reconstruction quality change as regularisation is increased. Running only the best-performing lambda does not reproduce the full experiment.

## 11. Iteration and stopping behaviour

The maximum iteration count is 120 and the requested tolerance is 1e-6. The dissertation reports that the recorded CT runs reached the iteration limit, with a convergence rate of zero. This distinction should be retained when interpreting the results: 120 iterations is an imposed computational limit, not evidence that all four methods reached the mathematical solution to the requested tolerance.

Changing the maximum iteration count can substantially change runtime and reconstruction quality. It therefore constitutes a change to the benchmark.

## 12. Output restrictions

The numerical outputs are generated rather than being required repository inputs. CT.py creates CT_solver_results.csv, CT_solver_summary.csv, CT_solver_runtime_comparison.csv and CT_experiment_metadata.txt. plot_CT.py creates the CT_plots directory containing the PNG figures and plot_summary.txt.

It is reasonable to remove generated CSV and PNG files from the repository to reduce repository size. A future researcher can regenerate them from the two supplied Python files. If generated outputs are removed, the benchmark configuration and documentation should remain unchanged so that the outputs can be recreated.

## 13. Timing and hardware reporting

Runtime is not a hardware-independent quantity. A reproduction report should record at least the machine or cluster used, CPU allocation, memory allocation, Python version, relevant package versions, and whether the calculation was run locally or through Slurm.

The dissertation's reported runtime values should be used as reference results rather than as universal performance guarantees. A different machine may produce faster or slower timings while still reproducing the mathematical experiment correctly.

## 14. What counts as an exact reproduction

For this repository, an exact computational reproduction means retaining the supplied problem dimensions, number of views, matrix-free operator, phantom construction, noise procedure and seed, regularisation values, solver definitions, tolerance, maximum iteration count and number of repeats. It also means running the same two-stage workflow: numerical benchmark first, plotting second.

A run with different image dimensions, fewer views, different noise, different lambda values, a different stopping limit, a different preconditioner or an explicitly formed CT matrix may still be scientifically useful, but it is a modified experiment rather than an exact reproduction.

## 15. Important limitations

The supplied CT operator is a synthetic parallel-beam model. It is not a clinical scanner model and does not claim to reproduce a particular patient's acquisition geometry. The phantom provides a controlled reference image so that reconstruction error can be measured.

The reported timings are also environment-dependent. They should not be interpreted as a universal ranking of the four algorithms. The dissertation uses the experiment to compare their behaviour under the defined few-view, matrix-free conditions.

Finally, the benchmark does not hard-code a preferred winner.